

Lisbon Airport

This Quarto document lists the data preparatory action for integrating Lisbon airport, LPPT. The APDF data for LPPT was extracted from ECTRL data warehouse and stored locally. This script is not reproducible if the ADPF data are not accessible.

```
library(tidyverse)
```

```
— Attaching core tidyverse packages ————— tidyverse 2.0.0
—
✓ dplyr      1.1.4      ✓ readr      2.1.5
✓ forcats    1.0.0      ✓ stringr    1.5.1
✓ ggplot2     3.5.1      ✓ tibble     3.2.1
✓ lubridate  1.9.4      ✓ tidyr      1.3.1
✓ purrr       1.0.2
— Conflicts ————— tidyverse_conflicts()
—
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
source(here::here("R", "rqutils-zip.R"))
source(here::here("R", "rq-apdf-to-punc.R"))
source(here::here("R", "rq-apdf-to-pbwg.R")) # todo- clean - need to load
punc first.
source(here::here("R", "rq-apdf-asma-dsma.R"))
source(here::here("R", "03-traffic_characterization-graphs.R"))
```

Attaching package: 'flextable'

The following object is masked from 'package:purrr':

compose

Attaching package: 'zoo'

The following objects are masked from 'package:base':

as.Date, as.Date.numeric

Attaching package: 'magrittr'

The following object is masked from 'package:purrr':

set_names

The following object is masked from 'package:tidyr':

extract

Using poppler version 23.04.0

Loading required package: usethis

Attaching package: 'arrow'

The following object is masked from 'package:magrittr':

is_in

The following object is masked from 'package:lubridate':

duration

The following object is masked from 'package:utils':

timestamp

Rows:	2192	Columns:	3—	Column	specification
-------	------	----------	----	--------	---------------

Delimiter: ","

dbl (2): DLY_FLTS, MVTS_NORM_ROLLAVG

date (1): DATE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Rows:	2192	Columns:	4—	Column	specification
-------	------	----------	----	--------	---------------

Delimiter: ","

chr (1): REG

dbl (2): FLTS, YEAR

date (1): DATE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Rows:	60	Columns:	3—	Column	specification
-------	----	----------	----	--------	---------------

Delimiter: ","

chr (1): ICAO

```
dbl (2): YEAR, TOT_FLTS_YEAR
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.Rows: 716 Columns: 4— Column specification
_____
Delimiter: ","
chr (1): ICAO
dbl (3): YEAR, MONTH, TOT_FLTS_MONTH
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.New names:
```

```
lppt_zip <- list.files(pth_apdf, pattern = "^LPPT-apdf")
whats_there <- check_zip_content(pth_apdf, lppt_zip)
whats_there
```

	Name	Length	Date
1	LPPT_01-JAN-2019_31-DEC-2019.gz.parquet	47842469	2025-04-04 15:44:00
2	LPPT_01-JAN-2020_31-DEC-2020.gz.parquet	21018298	2025-04-04 15:43:00
3	LPPT_01-JAN-2021_31-DEC-2021.gz.parquet	26818339	2025-04-04 15:43:00
4	LPPT_01-JAN-2022_31-DEC-2022.gz.parquet	44720672	2025-04-04 15:42:00
5	LPPT_01-JAN-2023_31-DEC-2023.gz.parquet	48624270	2025-04-04 14:05:00
6	LPPT_01-JAN-2024_31-DEC-2024.gz.parquet	47881436	2025-04-04 14:03:00

Airport Traffic

For BRA-EUR, we use PBWG daily traffic stats as analytic data.

```
# let's iterate over each year
whats_there <- whats_there |> mutate(YEAR = str_sub(Name, 13,16))

# loop for one
read_one_year <- function(.pth_apdf, .archive, .yr, ...){
  this_target <- whats_there |> dplyr::filter(YEAR == .yr)
  this_data <- read_zip(.pth_apdf, .archive, .files = this_target$Name)
  # this_data |> read_and_write_apr_tfc("LPPT", .yr)
}

# test-run: read_one_year(pth_apdf, lppt_zip, .yr = 2019) |> print()
# do all
whats_there$YEAR |> purrr::walk(.f = ~ read_one_year(pth_apdf, lppt_zip, .x))
```

- document write out of traffic (count) data set
- script changed to

Check what we have - traffic data set

```
tfc_data <- list.files(here::here("data-prep-eur"), pattern = "apt-tfc-LPPT",
full.names = T) |>
  purrr::map(.f = ~ readr::read_csv(.x, show_col_types = F))
```

```
lppt_annual_tfc <- tfc_data |> bind_rows() |>
  mutate(YEAR = lubridate::year(Date)) |>
  group_by(ICA0, YEAR) |>
  reframe(MVTS = sum(ARRS) + sum(DEPS))
```

```
lppt_annual_tfc |> gt::gt()
```

ICA0	YEAR	MVTS
LPPT	2019	221787
LPPT	2020	90080
LPPT	2021	116087
LPPT	2022	203161
LPPT	2023	227381
LPPT	2024	229900

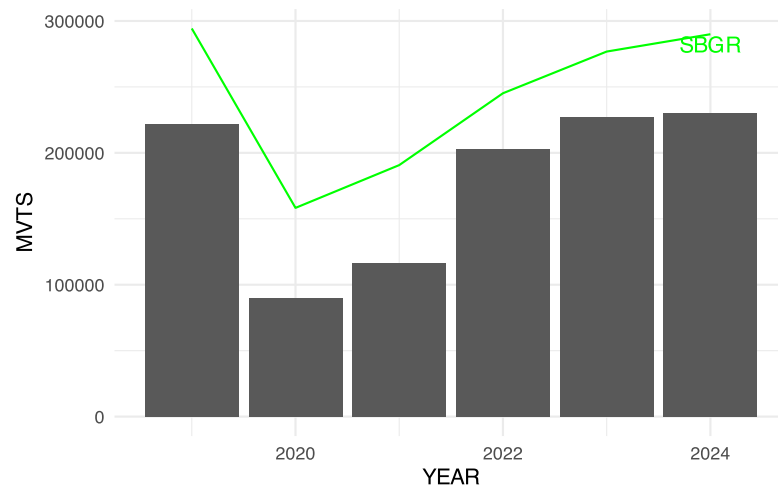
```
tfc_apt_bra <- read_csv("../data/BRA-airport-tfc.csv", show_col_types = FALSE)
```

New names:

- `` -> `...1`

```
sbgr_tfc_annual <- tfc_apt_bra |> filter(ICA0 %in% "SBGR") |> mutate(YEAR =
lubridate::year(Date)) |> group_by(ICA0, YEAR) |> reframe(MVTS = sum(ARRS) +
sum(DEPS))
```

```
lppt_annual_tfc |> ggplot() +
  geom_col(aes(y = MVTS, x = YEAR)) +
  geom_line(data = sbgr_tfc_annual, aes(y = MVTS, x = YEAR, group = ICA0), color
= "green" ) +
  geom_text(
    data = sbgr_tfc_annual |> filter(YEAR == 2024)
    , aes(y = MVTS, x = 2024, label = ICA0)
    , color = "green"
    , vjust = 1.2 )
```



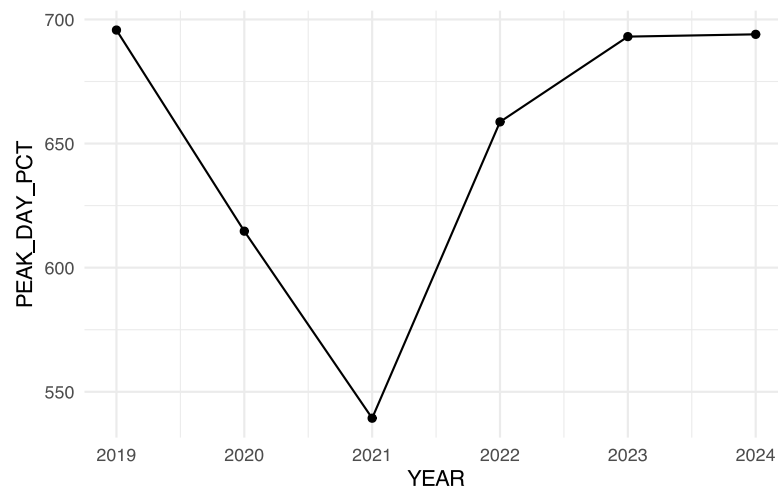
Airport Peak Traffic

based on BRA-EUR, i.e. 99th percentile

the peak day (99th percentile) is calculated from the daily traffic stats

```
lppt_annual_peak <- tfc_data |> bind_rows() |> peak_day_from_counts() |>
  add_nbr_rwy() |>
  mutate(REGION = "EUR")

lppt_annual_peak |>
  ggplot(aes(x = YEAR, y = PEAK_DAY_PCT)) +
  geom_line(aes(group = ICAO)) +
  geom_point()
```



Airport Punctuality

```

lppt_adpf <- read_zip(pth_apdf, lppt_zip)

# do it for one
calc_punc_for_df_with_delay <- function(.df){
  this_punc <- .df |>
  prep_apdf() |>
  distinct() |>
  mutate(ICA0 = "LPPT") |>
  add_delay_and_dlygrp() |>
  package_pbwg_punc_analytic()
  return(this_punc)
}

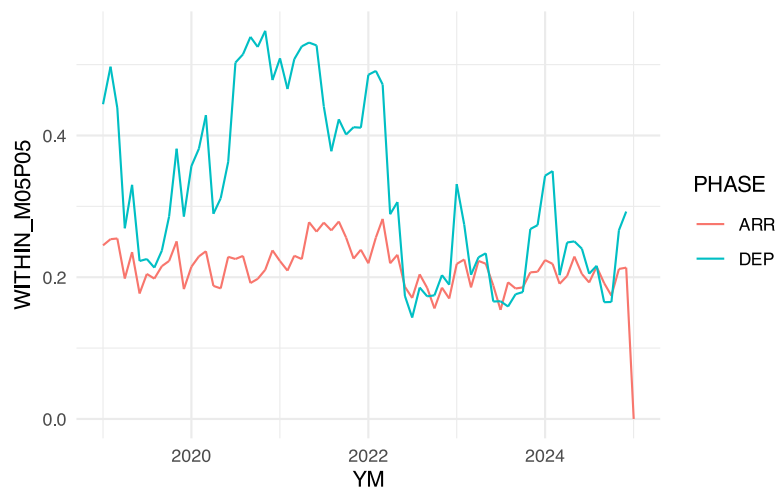
punc_2019 <- lppt_adpf[[1]] |> calc_punc_for_df_with_delay()

# iterate over all
punc_lppt <- lppt_adpf |>
  purrr::map(.f = ~ .x |> calc_punc_for_df_with_delay())

# write out --- punctuality data for LPPT
# punc_lppt |> bind_rows() |>
# readr::write_csv(here::here("data", "PBWG-EUR-PUNC-LPPT-2019-2024.csv"))

```

check what we have



i Note

CHECK FOR DATA IN 2nd Half of 2024

Data file has full 2024 – check data types – why does function fail?

Additional Taxi-Time

Per PBWG convention, we determine the reference times for the year 2024.

```
source(here::here("R", "rq-apdf-to-txxt.R"))
```

```
lppt_2024 <- read_zip(pth_apdf, lppt_zip, "LPPT_01-JAN-2024_31-
DEC-2024.gz.parquet")
txxt_ref_2024 <- lppt_2024 |>
  rename(PHASE = SRC_PHASE, RWY = AP_C_RWY, STND = AP_C_STND) |>
  prepare_eur_txxts() |>
  calc_pct20_cases(ymd_hms("2024-01-01 00:00:00"), ymd_hms("2024-12-31
24:00:00")) |>
  rename(N_2024 = N, REF_20_2024 = REF_20)

# compare with 2019
lppt_2019 <- read_zip(pth_apdf, lppt_zip, "LPPT_01-JAN-2019_31-
DEC-2019.gz.parquet")
txxt_ref_2019 <- lppt_2019 |>
  rename(PHASE = SRC_PHASE, RWY = AP_C_RWY, STND = AP_C_STND) |>
  prepare_eur_txxts() |>
  calc_pct20_cases(ymd_hms("2019-01-01 00:00:00"), ymd_hms("2019-12-31
24:00:00")) |>
  rename(N_2019 = N, REF_20_2019 = REF_20)

# correct for runway change
txxt_ref_2019 <- txxt_ref_2019 |>
  mutate( RWY_OLD = RWY
    , RWY      = case_when(
      RWY %in% "03" ~ "02"
    , RWY %in% "21" ~ "20"
    )
  )
```

```
txxt_ref_2024 |> full_join(txxt_ref_2019, by = join_by(PHASE, RWY, STND)) |> # mutate(DELTA
= REF_20_2024 - REF_20_2019) |> write_csv(here::here("data-prep-eur", "apt-txxt-ref-LPPT.csv"))
```

```
# load reference taxi times for LPPT
lppt_txxt_refs <- read_csv(
  here::here("data-prep-eur", "apt-txxt-ref-LPPT.csv")
  , show_col_types = FALSE) |>
# force ref time for all cases -----
mutate(REF_20_2024 = case_when(
  is.na(REF_20_2024) & !is.na(REF_20_2019) ~ REF_20_2019
  , .default = REF_20_2024)
) |>
select(PHASE, RWY, STND, REF = REF_20_2024)
```

```
# load all movement data 2019-2024
lppt_txxts <- read_zip(pth_apdf, lppt_zip) |> bind_rows() |>
  rename(PHASE = SRC_PHASE, RWY = AP_C_RWY, STND = AP_C_STND) |>
  mutate(ICA0 = "LPPT", .before = PHASE) |>
  prepare_eur_txxts() |>
  # account for runway change
  mutate(RWY = case_when(
    RWY %in% "03" ~ "02"
    , RWY %in% "21" ~ "20"
  )
)
```

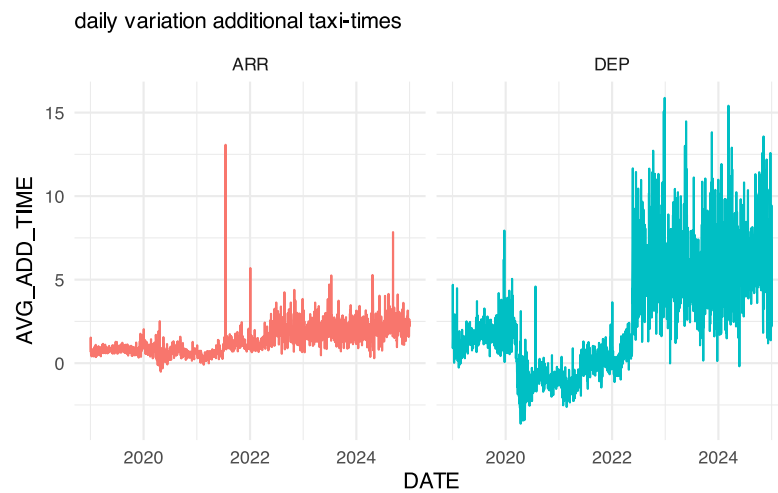
```
lppt_txxts <- lppt_txxts |>
  left_join(lppt_txxt_refs, by = join_by(PHASE, RWY, STND)) |>
  mutate(ADD_TXXT = TXXT - REF)
```

```
Warning in left_join(lppt_txxts, lppt_txxt_refs, by = join_by(PHASE, RWY, :
Detected an unexpected many-to-many relationship between `x` and `y`.
i Row 15404 of `x` matches multiple rows in `y`.
i Row 12 of `y` matches multiple rows in `x`.
i If a many-to-many relationship is expected, set `relationship =
  "many-to-many"` to silence this warning.
```

```
lppt_txxt_add_times <- lppt_txxts |>
  mutate(ICA0 = "LPPT", DATE = date(BLOCK_TIME_UTC)) |>
  group_by(ICA0, DATE, PHASE) |>
  reframe( MVTs      = sum(!is.na(ADD_TXXT))
    , TXXT      = sum(TXXT, na.rm = TRUE)
    , ADD_TIME  = sum(ADD_TXXT, na.rm = TRUE)
  ) |>
  mutate(AVG_ADD_TIME = ADD_TIME / MVTs)
```

```
lppt_txxt_add_times |> mutate(PHASE = ifelse(PHASE == "ARR", "TXIT", "TXOT")) |>
write_csv(here::here("data","EUR-txxt-LPPT.csv"))
```

```
lppt_txxt_add_times |>
  ggplot() +
  geom_path(aes(x = DATE, y = AVG_ADD_TIME, color = PHASE, group = PHASE)) +
  facet_wrap(~ PHASE) +
  labs(subtitle = "daily variation additional taxi-times") +
  theme(legend.position = "none")
```



Additional Time in Terminal Airspace

BRA-EUR focusses on ASMA (KPI08) - additional time in the arrival airspace.

```
# load the data sets
if(!exists("lppt_adpf")){
  lppt_adpf <- read_zip(pth_apdf, lppt_zip)
}
```

ICAO,DOF,ARRS,ARRS40,ARRS100,TOT_A40,TOT_REF40,ADD40,TOT_A100,TOT_REF100,AD-
D100,AVG_ADD40,AVG_ADD100